Método de la Bisección

Tema 2. Ecuaciones lineales y no lineales

Cálculo Numérico
Diego Rojo

Ecuaciones lineales y no lineales

Ecuaciones lineales y no lineales

- Problemas científicos a menudo requieren resolver ecuaciones de la forma:

$$f(x) = 0$$

- Una solución (*raíz*) es un valor x^* que satisface $f(x^*) = 0$.

Métodos Numéricos

- Desafortunadamente, solo podemos resolver analíticamente ecuaciones lineales y ciertas no lineales.
  - Si f es lineal, $f(x) = ax + b$, la solución solo requiere una división.
  - Si f es polinómica, $f(x) = a_n x^n + \cdots + a_1 x + a_0$, el teorema de Abel-Ruffini nos dice que no hay fórmula algebraica general para encontrar las raíces de polinomios de grado 5 o superior.
  - Si f **no** es polinómica, con más razón, no hay fórmula general para encontrar las raíces.
- En este tema nos centraremos en **métodos numéricos** para aproximar soluciones de ecuaciones generales $f(x) = 0$.

Derivación

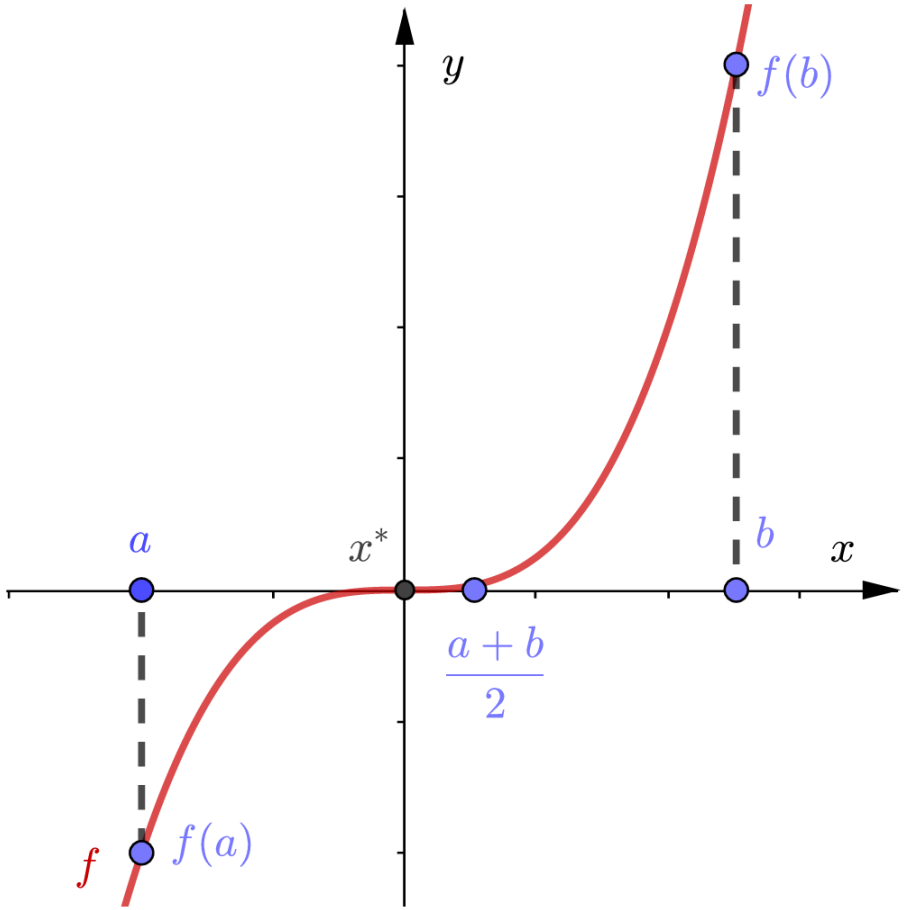
Método de Bisección

- El método de la bisección es un método numérico básico para aproximar raíces de ecuaciones $f(x) = 0$.
- **Requisitos:**
  - Función continua $f(x)$.
  - Intervalo inicial $[a, b]$ donde $f(a) \cdot f(b) < 0$ (cambio de signo).
- **Convergencia:** Siempre converge, aunque es relativamente lento.
- Se basa en el **Teorema de Bolzano**.

Teorema de Bolzano

> **Teorema de Bolzano**
> Si una función $f(x)$ es continua en un intervalo cerrado $[a, b]$ y $f(a) \cdot f(b) < 0$, entonces existe al menos un punto $c \in (a, b)$ tal que $f(c) = 0$.

Punto Medio



Proceso Iterativo

1. **Punto medio:** Calcular el punto medio del intervalo $[a, b]$:

$$x_0 = \frac{a+b}{2}$$

2. **Nuevo intervalo:**
   - Si $f(a) \cdot f(x_0) < 0$, la raíz está en $[a, x_0]$. El nuevo intervalo es $[a, x_0]$.
   - Si $f(x_0) \cdot f(b) < 0$, la raíz está en $[x_0, b]$. El nuevo intervalo es $[x_0, b]$.
   - Si $f(x_0) = 0$, entonces x_0 es la raíz.

- Repitiendo este proceso generamos una **secuencia de aproximaciones** $x_k \approx x^*$
- App interactiva

Criterio de Parada

- En la práctica, necesitamos **detener las iteraciones** cuando el valor de x_k está **suficientemente cerca** de la raíz x^*.
    - **Problema**: Normalmente, desconocemos x^*, luego no podemos calcular el error $|x_k - x^*|$.
- **Criterio de parada:** detener el proceso cuando dos aproximaciones sucesivas estén **suficientemente cerca**:

$$|x_k - x_{k-1}| < \mathrm{TOL},$$

donde TOL es una tolerancia predefinida.

Justificación (Criterio de Parada)

- Si la secuencia $\{x_k\}_{k\in\mathbb{N}}$ converge a x^*, entonces para valores de $k > K$ (para algún $K \in \mathbb{N}$), el error $|x_k - x^*| < \epsilon$.
- Esta convergencia implica que la **diferencia entre aproximaciones sucesivas** $|x_{k+1} - x_k|$ también se hará pequeña para valores de $k > K$.

$$|x_{k+1} - x_k| = |x_{k+1} - x^* + x^* - x_k| \leq |x_{k+1} - x^*| + |x_k - x^*| < 2\epsilon.$$

Criterios de Parada Alternativos

Además del criterio principal $|x_k - x_{k-1}| < \mathrm{TOL}$, existen criterios de parada alternativos:

- **Valor de la función**: Comprobar si la magnitud de $f(x_k)$ es cercana a cero:

$$|f(x_k)| < \mathrm{TOL}_f$$

- **Longitud del intervalo** (Solo Bisección): Detener cuando el intervalo $[a, b]$ sea suficientemente pequeño:

$$|b - a| < \mathrm{TOL}_{interval}$$

- *Nota: Para métodos que no converjan siempre, es crucial fijar además un número máximo de iteraciones (* `MAXIT` *) para evitar bucles infinitos.*

Algoritmo: Método de la Bisección

Algorithm 11 Bisection method

```
Set a tolerance TOL for accuracy
Initialize the interval [a, b]
Set k = 0
Check if f(a) · f(b) < 0 (if not then quit)
while |b − a| > TOL do
    Set x_k = (a + b)/2 (the new approximation)
    Check in which interval [a, x_k] or [x_k, b] the function changes sign
    if f(x_k) · f(a) < 0 then
        b = x_k
    else if f(x_k) · f(b) < 0 then
        a = x_k
    else
        Break the loop because f(x_k) = 0
    end if
    k = k + 1
end while
Return the approximation of the root x*
```

Implementación

Implementa la siguiente función en Python:

```
def biseccion(f, a, b, tol = 1.e-6, verbose = False):
```

Nota: Puedes encontrar una implementación del método de la bisección en `scipy.optimize.bisect`.

```
bisect(f, a, b, args, xtol, rtol, maxiter, full_output, disp)
```

Ejemplo: $ln(x) + x = 0$

- Resolver la ecuación $ln(x) + x = 0$ en el intervalo $[0.1, 1]$.

```
def f(x):
    y = np.log(x) + x
    return y

a = 0.1
b = 1.0
tol = 1.e-4

x_root = biseccion(f, a, b, tol)

if x_root is not None:
    print('xk: ', x_root)
    print('f(xk): ', f(x_root))
```

Convergencia

Convergencia

- Queremos saber si las aproximaciones generadas **convergen a la solución correcta**.
- También queremos saber **cuán rápido** convergen.
- Decimos que un método converge si

$$\lim_{k\to\infty} |e_k| = 0.$$

donde $e_k = x_k - x^*$ es el error en la iteración k.

Convergencia del Método de Bisección

- El método de la bisección siempre converge a una raíz si se cumplen las condiciones iniciales.
- **Error de aproximación:** $|e_k| = |x_k - x^*|$.

> **Proposición (Convergencia)**
>
> Si $f \in C[a,b]$ y $f(a) \cdot f(b) < 0$, entonces la secuencia $\{x_k\}_{k\in\mathbb{N}}$ generada por el método de la bisección converge a la raíz x^* de $f(x) = 0$ o existe un k tal que $x_k = x^*$. Además,
>
> $$|x_k - x^*| \leq \frac{b-a}{2^{k+1}}, \quad k = 0, 1, 2, \ldots$$

> *Demostración: Computational Mathematics (Dimitrios Mitsotakis) — Proposición 5.1*

Estimación del Número de Iteraciones

- El resultado anterior, nos permite además estimar el número máximo de iteraciones necesarias para alcanzar una precisión deseada.

> **Corolario (Número de Iteraciones)**
>
> Sea $f \in C[a,b]$ y $f(a) \cdot f(b) < 0$ y sea $\{x_k\}_{k\in\mathbb{N}}$ la secuencia generada por el método de la bisección. Entonces, para cualquier $\epsilon > 0$, si
>
> $$k \geq N = \left\lceil \frac{\ln(b-a) - \ln(2\epsilon)}{\ln(2)} \right\rceil$$
>
> se tiene que $|e_k| = |x_k - x^*| \leq \epsilon$.

Ejemplo: Número de Iteraciones

- Intervalo inicial $[0, 1]$ y $\epsilon = 10^{-4}$.

$$N = \left\lceil \frac{\ln(1-0) - \ln(2 \cdot 10^{-4})}{\ln(2)} \right\rceil = \left\lceil \frac{\ln(1) - \ln(2 \cdot 10^{-4})}{\ln(2)} \right\rceil = \lceil 12.29 \rceil = 13$$

- Necesitamos 14 iteraciones para asegurar la precisión deseada.

Orden de Convergencia

Orden de Convergencia

- Tras verificar la convergencia, queremos saber **cuán rápido** el método se acerca a la solución x^*.
- La **velocidad de convergencia** se describe mediante el **orden de convergencia** (r).
- El orden de convergencia r para una secuencia de errores $|e_k| = |x_k - x^*|$ es el número $r > 1$ para el cual existe una constante $C > 0$ tal que:

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^r} = C.$$

Interpretación de la Orden de Convergencia

- Si $|e_k| < 1$ (error pequeño), entonces $|e_k|^r < |e_k|$ para $r > 1$.
- Un orden $r > 1$ implica que el error en la siguiente iteración $|e_{k+1}|$ es **mucho menor** que el error actual $|e_k|$.
  - La magnitud de la reducción del error está determinada por el exponente r.
  - $|e_{k+1}| \approx C|e_k|^r < |e_k|$.

Tipos de Convergencia

- **Convergencia Lineal** ($r = 1$): $|e_{k+1}| \approx C|e_k|$ con $C < 1$.
- **Convergencia Cuadrática** ($r = 2$): $|e_{k+1}| \approx C|e_k|^2$.

Convergencia del Método de Bisección

> **Corolario (Convergencia Lineal)**
>
> Sea $f \in C[a,b]$ y $f(a) \cdot f(b) < 0$ y sea $\{x_k\}_{k\in\mathbb{N}}$ la secuencia generada por el método de la bisección. Entonces, para todo $k \geq 0$,
>
> $$|e_{k+1}| \leq \frac{1}{2}|e_k|\,,$$
>
> donde $e_k = x_k - x^*$ es el error en la iteración k.

- Es decir, el método de bisección tiene *al menos* **convergencia lineal** (orden $r = 1$) con constante $C = 1/2$.

Estimación Experimental

- Si no podemos obtener una estimación teórica, podemos aproximar el orden de convergencia **experimentalmente**.
- Calculamos los errores en tres iteraciones sucesivas, e_1, e_2, e_3.
- Asumimos que $|e_2| \approx C|e_1|^r$ y $|e_3| \approx C|e_2|^r$.
- Dividiendo ambas expresiones, obtenemos:

$$\frac{|e_2|}{|e_3|} \approx \left(\frac{|e_1|}{|e_2|}\right)^r$$

- Despejando r:

$$r \approx \frac{\ln(|e_2|/|e_3|)}{\ln(|e_1|/|e_2|)}.$$